

Analysis of the Implementation of Deep Convolutional Generative Adversarial Networks

Thadar Soe¹

¹Pusan National University

1. Introduction

In this implementation, a Deep Convolutional Generative Adversarial Network (DCGAN) was implemented to investigate its generative performance and training behavior on the CelebA[3] face dataset. The model follows the architectural principles established in the original GAN paper[2], including the use of transposed convolutions for upsampling, stride-2 convolutions in the discriminator, Batch Normalization in both networks, and ReLU or LeakyReLU activations depending on the direction of information flow. The latent vector was sampled from a 100-dimensional Gaussian distribution and mapped through a sequence of upconvolutions to synthesize 64×64 RGB face images. The foundational concepts and architectural variants of GANs have been extensively reviewed in recent literature [1].

2. Implementation

The architectural guidelines proposed for stable Deep Convolutional GANs[4] play an essential role in ensuring reliable adversarial training. Replacing pooling layers with strided convolutions (and fractional-strided convolutions in the generator) allows the model to learn spatial downsampling and up-sampling directly, preserving important structure that fixed pooling tends to discard.

For the DCGAN implementation, the CelebA dataset[3] was used as the training corpus, and all images were resized to 64×64 to match the architectural specifications of the model. The network was constructed following the established DCGAN architectural guidelines, which replace pooling layers with strided or fractional-strided convolutions, employ Batch Normalization throughout both networks, remove fully connected layers, and adopt ReLU activations in the generator and LeakyReLU in the discriminator.

2.1. Generator Implementation

The generator is constructed as a sequence of five transposed convolutional layers that progressively upsample a 100-dimensional latent vector into a 64×64 RGB image. Each layer follows the DCGAN architectural guidelines, including the use of Xavier initialization, Batch Normalization in all but the final layer, and ReLU activations throughout the network except for the output, which uses a Tanh activation.

- **Layer 1:** Expands the latent vector from $(bs, 100, 1, 1)$ to $(bs, 512, 4, 4)$ using a kernel size of 4, stride 1, and padding 0.
- **Layer 2:** Upsamples the feature map from 4×4 to 8×8 and reduces channels from 512 to 256.
- **Layer 3:** Further upsamples the representation from 8×8 to 16×16 with channels decreasing from 256 to 128.
- **Layer 4:** Expands the spatial resolution from 16×16 to 32×32 and reduces channels from 128 to 64.
- **Layer 5 (Output Layer):** Produces the final $(bs, 3, 64, 64)$ RGB output. Batch Normalization is omitted, and Tanh is used to map values to $[-1, 1]$.

Overall, the generator progressively transforms a small latent vector into a high-resolution image by learning spatial structure at multiple scales, following the canonical DCGAN design.

The forward function of the generator defines the flow of data through the five transposed convolutional layers. The input is the latent vector $z \in R^{100}$, reshaped to $(bs, 100, 1, 1)$, which is then progressively upsampled into

a 64×64 RGB image. Each layer applies a learned upconvolution, followed by Batch Normalization and ReLU activation, except for the final layer which uses a Tanh activation. The implementation is as follows:

- **Input:** latent tensor x of shape $(bs, 100, 1, 1)$.
- **Layer sequence:**

```
x = self.upconv1(x)
x = self.upconv2(x)
x = self.upconv3(x)
x = self.upconv4(x)
x = self.upconv5(x)
```
- **Output:** a synthesized RGB image of shape $(bs, 3, 64, 64)$.

At each stage, the spatial resolution is doubled while the channel dimension is reduced, enabling the generator to construct images in a coarse-to-fine manner. The final Tanh activation ensures that the output values lie within the normalized range $[-1, 1]$, consistent with the preprocessing applied to the CelebA dataset.

2.2. Discriminator Implementation

The discriminator mirrors the generator's structure by using a sequence of five strided convolutional layers that progressively reduce the spatial resolution of a 64×64 RGB input image. Following the DCGAN architectural guidelines, the discriminator replaces all pooling operations with strided convolutions, applies Batch Normalization to every layer except the first and last, and employs LeakyReLU activation throughout the network to ensure stable gradient flow during adversarial training.

- **Layer 1:** Processes the input image $(bs, 3, 64, 64)$ and reduces the spatial resolution to $(bs, 64, 32, 32)$. Batch Normalization is omitted in accordance with DCGAN guidelines, and LeakyReLU is used to prevent inactive neurons.
- **Layer 2:** Further reduces spatial size from 32×32 to 16×16 and increases channel depth from 64 to 128. Batch Normalization stabilizes the activation distribution.
- **Layer 3:** Downsamples the representation to 8×8 while expanding channels from 128 to 256, enabling the discriminator to learn increasingly abstract features for real/fake classification.
- **Layer 4:** Reduces spatial resolution from 8×8 to 4×4 with a channel depth of 512, forming the deepest hierarchical representation used for discrimination.
- **Layer 5 (Output Layer):** Produces the final output of shape $(bs, 1, 1, 1)$, representing the probability that the input image is real. Batch Normalization is omitted, and a Sigmoid activation is used for binary classification.

Overall, the discriminator compresses the input image through a hierarchy of learned convolutional filters, progressively capturing low-level textures and high-level structures necessary to distinguish authentic CelebA samples from generated ones. The use of LeakyReLU throughout the network ensures non-vanishing gradients, while the strided convolutions preserve spatial coherence without the need for pooling operations.

The forward function processes the input image sequentially through all five convolutional layers, applying the appropriate normalization and activation functions at each stage. The implementation is as follows:

- **Input:** tensor x of shape $(bs, 3, 64, 64)$.

- **Layer sequence:**

```
x = self.conv1(x)
x = self.conv2(x)
x = self.conv3(x)
x = self.conv4(x)
x = self.conv5(x)
```

- **Output:** probability tensor of shape (bs, 1, 1, 1).

At each stage, the discriminator reduces the spatial resolution while increasing representational depth, allowing it to extract hierarchical visual features relevant for adversarial training.

3. Results



Figure 1: Generated images at Epochs 1–5.

The progression of generated samples across training epochs shows a clear and stable convergence pattern characteristic of DCGAN training. In the earliest stage (far-left), the generator produces highly noisy outputs with no distinguishable facial structure. The coarse color blobs and rough spatial patterns indicate that the network is beginning to learn but has not yet captured any semantic structure. As training progresses, facial features such as eyes, noses, mouths, and hair outlines become increasingly distinct, exhibiting stronger structural consistency and improved sharpness. Nevertheless, occasional samples showcase artifacts such as asymmetric facial components, distorted backgrounds, and slight blending between features. In the final columns (rightmost), the generator produces high-quality and coherent face images with recognizable identity characteristics, well-defined eyes and hair textures, and improved color consistency. The images show significantly reduced noise and clearer feature separation.

Overall, the qualitative results indicate that the model successfully learned the underlying data distribution of CelebA. Despite minor imperfections, the final outputs demonstrate convincing realism and diversity, with the training progression showing gradual refinement from low-frequency structure to high-frequency facial details.

4. Citations

- [1] António Creswell, Tom White, Vincent Dumoulin, Suman Arulkumar, Ankita Sengupta, and Anil A Bharath. Generative adversarial networks: An overview. *IEEE Signal Processing Magazine*, 35(1):53–65, 2018.
- [2] Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial networks. *arXiv preprint arXiv:1406.2661*, 2014.
- [3] Ziwei Liu, Ping Luo, Xiaogang Wang, and Xiaoou Tang. Deep learning face attributes in the wild. *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pages 3730–3738, 2015.
- [4] Alec Radford, Luke Metz, and Soumith Chintala. Unsupervised representation learning with deep convolutional generative adversarial networks. *arXiv preprint arXiv:1511.06434*, 2015.